Fireflies: A Toolkit for Intrusion Tolerant Peer-to-Peer Overlay Network Services

Release 0.1.0

Håvard Johansen

August 6, 2007

haavardj@cs.uit.no

Abstract

Contents

1 Introduction **1**

2 Requirements **1**

3 Installation **2**

4 Usage **2**
- 4.1 Keys and Certificates . . . 2
- 4.2 The Group Object . . . 3
- 4.3 Basic Network Services . . . 3
- 4.4 Twisted . . . 3

1 Introduction

Fireflies is a toolkit for building scalable intrusion-tolerant peer-to-peer overlay-network services. The core component of Fireflies is a weakly consistent group membership protocol that provides to each member an up-to-date view of all group members. The membership protocol use redundancy to mask Byzantine fault such that malicious members cannot wrongly claim that live members have crashed, nor that crashed members are alive.

2 Requirements

The software required by Fireflies are listed below. The given version numbers are for those Fireflies is known to work. Other versions may or may not work.

Required software:

Python 2.5 (http://www.python.org)

Twisted Core 2.2 (http://twistedmatrix.com)

Python Cryptography Toolkit 2.0.1 (http://www.amk.ca/python/code/crypto)

NTL: A Library for doing Number Theory 5.4 (http://www.shoup.net/ntl)

GMP: GNU Multiple Precision Arithmetic Library 4.1.4 (http://gmplib.org/)

Optional software for the `fireflies.web` module:

Nevow 0.4.1 (http://nevow.com/)

Twisted Web 0.7.0 (http://twistedmatrix.com/)

Gnuplot 4.0 (http://www.gnuplot.info/) with build-in support for new-type PNG output.

Gnuplot.py 1.7 (http://gnuplot-py.sourceforge.net/)

Also, certain parts of Fireflies are implemented in a C++ extension. For this, you need the Python development headers and a C++ compiler.

3 Installation

After installing all the required packages, install Fireflies by:

1. Download 'fireflies-0.1.0.tar.gz' and untar the package.
2. In the 'fireflies-0.1.0/' directory, run `python setup.py install`.

This will install Fireflies into the standard Python site-packages path. You can specify a **–prefix=PATH** options to the `setup.py` script to install it in a different path.

4 Usage

4.1 Keys and Certificates

Each Fireflies group is associated with a group certificate. The group certificate embeds a public key and is self-signed with the corresponding private key. The holder of this key is the group's Certificate Authority (CA). To join a Fireflies group, a process must first obtain the group certificate and generate a private/public key pair and a member certificate. The process must have its MemberCertificate signed by the CA's private key. The following illustrates these operations:

```
from fireflies.state import Key, GroupCertificate, MemberCertificate

#At CA
caKey = Key() # Create Private key
caPublic = caKey.publicKey()  # Get public key
gc = GroupCertificate("foo", caPublic, ...)
caKey.sign(gc)

# At Member
myPrivateKey = Key()
myPublicKey = myPrivateKey.publicKey()
myCertificate = MemberCertificate(myPublicKey, myNetworkAddress)

# At CA
caKey.sign(myCertificate)
```

In addition to the above library functions, the included program **ffcekey** implements the basic CA functions.

4.2 The Group Object

Once in possession of a valid GroupCertificate and a MemberCertificate, a process may join the Fireflies group. The main interface to Fireflies is through the **fireflies.group.Group** class. Each group object has an attribute **view**, which points to a **fireflies.view.View** object. The view can be used to query and modify the group's state. The view holds references to **fireflies.state.Member** instances, one for each member. Fireflies implement strict rules regarding view changes. In particular, members can only be added indirectly by adding MemberCertificates. Also, a member can only be moved from a crashed state to a live state by adding a valid Note and a member can only be moved from a live state to a crashed state by adding a valid Accusation.

4.3 Basic Network Services

Fireflies members maintain up-to-date membership information by monitoring one-another and by exchanging MemberCertificates, Notes, and Accusations. Two network services are included: a failure detection mechanism based on UDP pinging, and a gossip service for exchange of membership events. To initialize these services the following must be executed:

```
from fireflies.group import Group
from fireflies.fd import FailureDetection
from fireflies.gossip import Gossip

group = Group(gc, myCertificate, myPrivateKey)
FailureDetection(group)
Gossip(group)
group.startService()
```

4.4 Twisted

Fireflies is built on top of the Twisted toolkit. Having some prior knowledge of Twisted is advisable before using Fireflies. At minimum, you must run the Twisted event loop in order to activate the Fireflies networking services. This is done in the following manner:

```
from twisted.internet import reactor

reactor.run()
```

Note that this command will block your main thread of execution.